

Python for Data Science

Buổi 8: Trực Quan Hóa Dữ Liệu I – Matplotlib Nền Tảng

PGS-TS Nguyễn Mạnh Hùng

August 19, 2026

Cấu trúc Buổi 8

- 1 Ôn tập Buổi 1–7 và đặt vấn đề
- 2 Module 29: Nhập môn Matplotlib
- 3 Module 30: Line Plots Module 31: Scatter Plots
- 4 Module 32: Error Bars Module 33: Density & Contour
- 5 Module 34: Histograms Module 35: Customizing Legends
- 6 Module 36: Customizing Colorbars Module 37: Multiple Subplots
- 7 Module 38: Text & Annotation Module 39: Ticks & Style Sheets
- 8 Bài tập áp dụng: births.csv

Lưu ý phạm vi

Buổi 8 tập trung Matplotlib nền tảng (Handbook Ch.4, phần đầu). Seaborn, 3D plotting, dữ liệu địa lý và EDA end-to-end sẽ học ở Buổi 9.

Nhắc lại Buổi 1–7

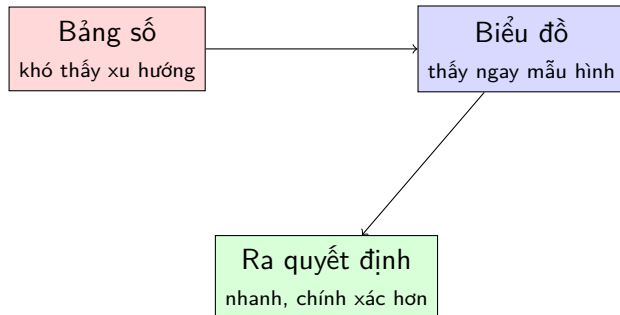
- Buổi 1–2: OOP nền tảng, NumPy (vector hóa, ufunc, broadcasting).
- Buổi 3–7: Pandas toàn diện – đọc dữ liệu, làm sạch, kết hợp, tổng hợp (groupby/pivot), chuỗi & thời gian.

Vấn đề còn lại

Suốt 7 buổi, kết quả phân tích chỉ được trình bày dưới dạng **bảng số** in ra màn hình. Nhưng bộ não con người nhận diện *hình ảnh* nhanh hơn bảng số rất nhiều – một biểu đồ tốt có thể truyền tải điều mà hàng trăm dòng số liệu không làm được.

Buổi 8 học công cụ trực quan hóa nền tảng nhất của hệ sinh thái Python: **Matplotlib** – thư viện mà cả Pandas `.plot()` lẫn Seaborn (Buổi 9) đều xây dựng bên trên.

Vì sao cần trực quan hóa dữ liệu?



Ví dụ kinh điển: bộ tứ dữ liệu Anscombe (1973) – 4 tập dữ liệu có *cùng* trung bình, phương sai, hệ số tương quan, nhưng hình dạng hoàn toàn khác nhau khi vẽ ra. Thống kê tóm tắt (Buổi 6) không thể thay thế việc *nhìn* vào dữ liệu.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Giải thích kiến trúc Figure và Axes của Matplotlib.
- 2 Phân biệt 2 giao diện lập trình: pyplot (trạng thái) và hướng đối tượng (OO).
- 3 Lưu hình vẽ ra file bằng `savefig()`.

Kiến trúc Figure và Axes

```
fig, ax = plt.subplots(2)      # fig: "to giấy vẽ"; ax: mang các "hệ trục" bên trong
ax[0].plot(x, np.sin(x))
ax[1].plot(x, np.cos(x))
```

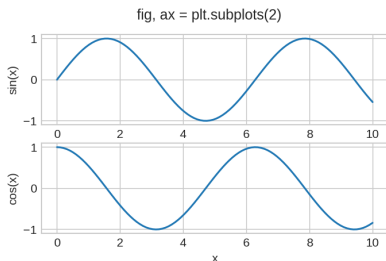


Figure là toàn bộ vùng vẽ (có thể chứa nhiều hệ trục); Axes là *một* hệ trục cụ thể (đường viền, tick, nhãn, và các đối tượng đồ họa bên trong) – tương tự quan hệ giữa DataFrame và từng cột Series (Buổi 3).

Hai giao diện: pyplot (trạng thái) và hướng đối tượng

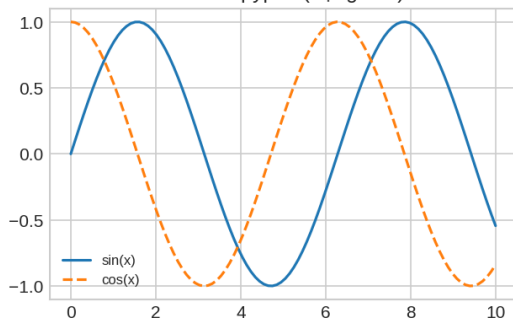
```
plt.plot(x, np.sin(x))  
plt.plot(x, np.cos(x))
```

Cách 1: pyplot -- "ve len truc hien hanh"

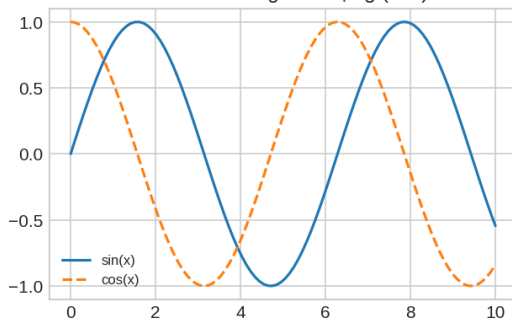
```
fig, ax = plt.subplots()  
ax.plot(x, np.sin(x))  
ax.plot(x, np.cos(x))
```

Cách 2: OO -- tao ax tuong minh, goi method tren ax

Cách 1: pyplot (trạng thái)



Cách 2: hướng đối tượng (OO)



Nên dùng giao diện nào?

Khuyến nghị

Với hình đơn giản, cả hai cách đều được. Nhưng khi cần **nhiều hệ trục** (subplots) hoặc **tùy biến sâu**, giao diện *hướng đối tượng* (`fig, ax = plt.subplots()` rồi gọi method trên `ax`) rõ ràng và ít nhầm lẫn hơn – đây cũng là phong cách được dùng xuyên suốt các module còn lại của Buổi 8.

- `fig.savefig("ten_file.png")`: lưu hình ra file (hỗ trợ png, pdf, svg, jpg...).
- `plt.style.use(...)`: đổi giao diện tổng thể (Module 39).
- `%matplotlib inline`: lệnh magic (Buổi 2) để hiển thị hình trực tiếp trong Jupyter.

Tổng kết Module 29

- Figure: toàn bộ vùng vẽ; Axes: một hệ trục cụ thể bên trong Figure.
- Giao diện pyplot (trạng thái, ngắn gọn) vs hướng đối tượng (`ax.<method>()`), rõ ràng hơn khi phức tạp).
- `fig.savefig()`: xuất hình ra file; hỗ trợ nhiều định dạng.
- Từ Module 30 trở đi, tài liệu ưu tiên phong cách hướng đối tượng.

Module 30: Line Plots (Đường)

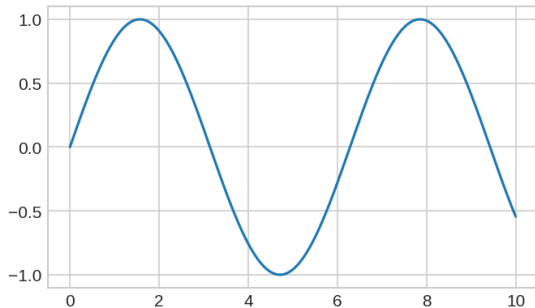
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Vẽ đường cơ bản bằng `ax.plot()`.
- 2 Tùy chỉnh màu sắc và kiểu nét vẽ.
- 3 Điều chỉnh giới hạn trục và tỉ lệ khung hình.
- 4 Gắn tiêu đề, nhãn trục, và chú giải (legend) đầy đủ cho một hình hoàn chỉnh.

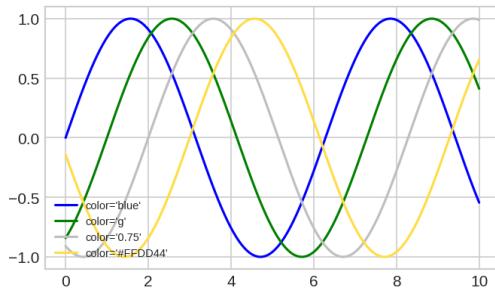
plot() cơ bản

```
fig, ax = plt.subplots()
x = np.linspace(0, 10, 1000)
ax.plot(x, np.sin(x));
```



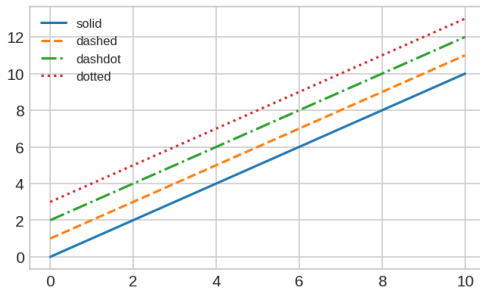
Tùy chỉnh màu sắc

```
ax.plot(x, np.sin(x - 0), color='blue')    # ten mau
ax.plot(x, np.sin(x - 1), color='g')      # ma mau ngan (rgbcmyk)
ax.plot(x, np.sin(x - 2), color='0.75')   # muc xam [0,1]
ax.plot(x, np.sin(x - 3), color='#FFDD44') # ma hex
```



Tùy chỉnh kiểu nét vẽ

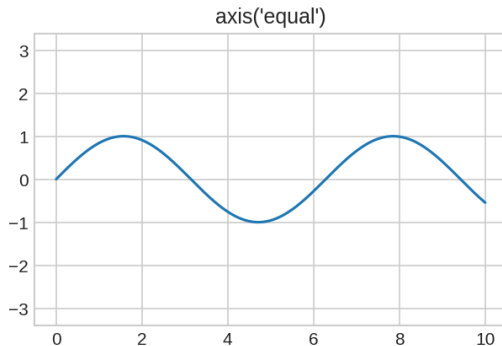
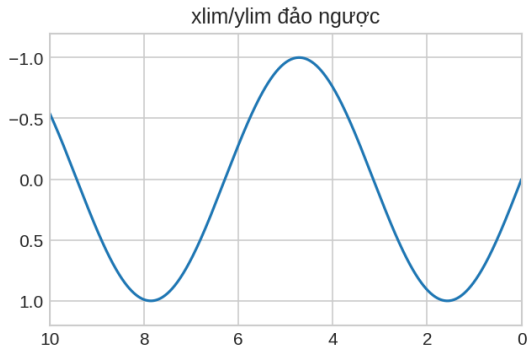
```
ax.plot(x, x + 0, linestyle='solid')  
ax.plot(x, x + 1, linestyle='dashed')  
ax.plot(x, x + 2, linestyle='dashdot')  
ax.plot(x, x + 3, linestyle='dotted')    # hoặc viết tắt: '-', '--', '-.', ':'
```



Giới hạn trục: xlim, ylim, axis

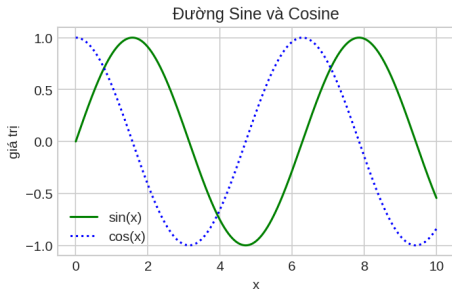
```
ax.set_xlim(10, 0)      # đảo ngược trục x  
ax.set_ylim(1.2, -1.2)  # đảo ngược trục y
```

```
ax.axis('equal')         # 1 đơn vị trục x = 1 đơn vị trục y (không bị méo hình)
```



Hoàn thiện: tiêu đề, nhãn trục, chú giải

```
ax.plot(x, np.sin(x), '-g', label='sin(x)')  
ax.plot(x, np.cos(x), ':b', label='cos(x)')  
ax.set_title("Đường Sine và Cosine")  
ax.set_xlabel("x"); ax.set_ylabel("giá trị")  
ax.legend();
```



Có thể gộp gọn bằng `ax.set(xlabel=..., ylabel=..., title=..., xlim=..., ylim=...)` – 1 lệnh thay vì nhiều lệnh `set_*` riêng lẻ.

Tổng kết Module 30

- `ax.plot(x, y)`: vẽ đường; ký hiệu tắt `'-g'`, `':b'` kết hợp kiểu nét + màu.
- `color=`, `linestyle=`: tùy chỉnh trực tiếp, hoặc qua ký hiệu tắt (format string).
- `ax.set_xlim/set_ylim`, `ax.axis('equal'/'tight')`: kiểm soát giới hạn/tỉ lệ trục.
- `ax.set_title/set_xlabel/set_ylabel`, `ax.legend()`: hoàn thiện hình để trình bày.

Module 31: Scatter Plots (Điểm rời rạc)

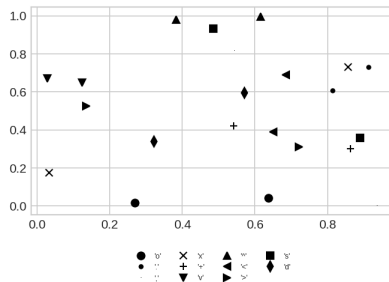
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Vẽ điểm rời rạc bằng `ax.plot()` (marker) và `ax.scatter()`.
- 2 Phân biệt hai hàm này về mặt hiệu năng và tính linh hoạt.
- 3 Mã hóa thêm 2 chiều dữ liệu qua màu sắc (`c=`) và kích thước (`s=`) điểm.
- 4 Áp dụng scatter plot đa biến trên một bộ dữ liệu thực tế (Iris).

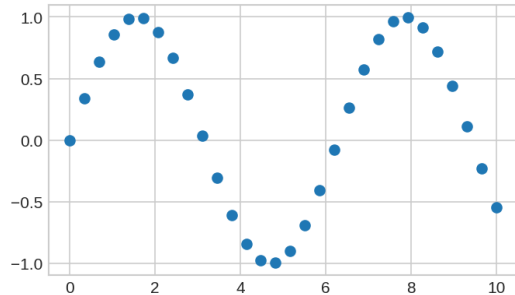
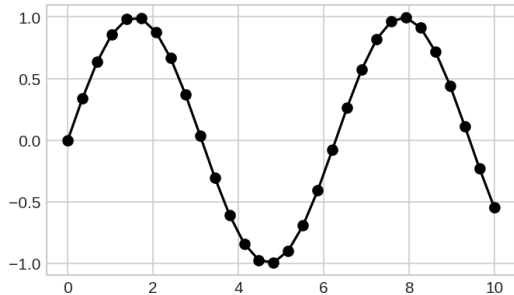
Các loại marker

```
for marker in ['o', '.', ',', 'x', '+', 'v', '^', '<', '>', 's', 'd']:  
    ax.plot(rng.random(2), rng.random(2), marker, color='black',  
            label=f"'{marker}'")
```



plot() với marker vs scatter()

```
ax.plot(x, y, '-ok');           # kết hợp đường nối + marker tròn, hiệu năng cao  
ax.scatter(x, y, marker='o');   # linh hoạt hơn (màu/kích thước riêng từng điểm)
```

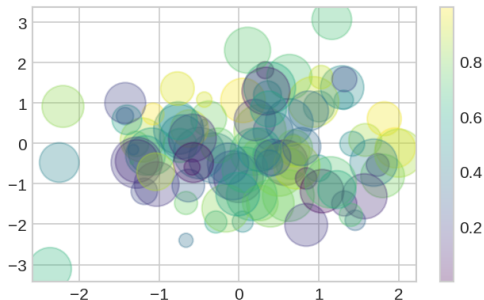


Khi nào dùng cái nào?

Dữ liệu *lớn*, cùng kiểu điểm: dùng `plot()` (nhanh hơn, 1 đối tượng duy nhất). Cần màu/kích thước riêng từng điểm: dùng `scatter()` (linh hoạt hơn).

Mã hóa thêm chiều dữ liệu: màu và kích thước

```
sc = ax.scatter(x, y, c=colors, s=sizes, alpha=0.3, cmap='viridis')  
fig.colorbar(sc, ax=ax)      # thanh màu tương ứng giá trị cột màu
```

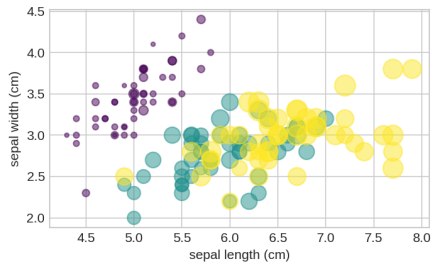


Mỗi điểm mang tới 4 chiều thông tin cùng lúc: vị trí (x, y), màu sắc (c), và kích thước (s) – kỹ thuật "bubble chart".

Ứng dụng thật: bộ dữ liệu Iris

```
from sklearn.datasets import load_iris
iris = load_iris()
features = iris.data.T          # 4 đặc trưng: sepal/petal length/width

ax.scatter(features[0], features[1], alpha=0.5,
           s=100 * features[3], c=iris.target, cmap='viridis')
ax.set_xlabel(iris.feature_names[0]); ax.set_ylabel(iris.feature_names[1])
```



150 mẫu hoa Iris (Fisher, 1936): trục x/y là dài/rộng đài hoa; kích thước điểm = rộng cánh hoa; màu = loài – 4 biến trong 1 hình 2D.

Tổng kết Module 31

- `ax.plot(..., marker)`: nhanh, phù hợp dữ liệu lớn cùng kiểu điểm.
- `ax.scatter()`: linh hoạt, mỗi điểm có thể khác màu/kích thước/alpha.
- `c=`, `s=`, `cmap=`, `alpha=`: mã hóa thêm chiều dữ liệu vào 1 hình 2D.
- Bộ Iris: ví dụ kinh điển cho scatter đa biến, sẽ gặp lại ở Machine Learning (Buổi 10–11).

Module 32: Error Bars (Thanh sai số)

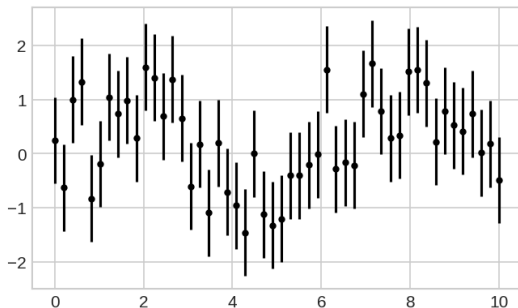
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Vẽ thanh sai số rời rạc bằng `ax.errorbar()`.
- 2 Tùy chỉnh kiểu hiển thị (màu, độ dày, capsize) của thanh sai số.
- 3 Vẽ dải bất định liên tục bằng `ax.fill_between()`.

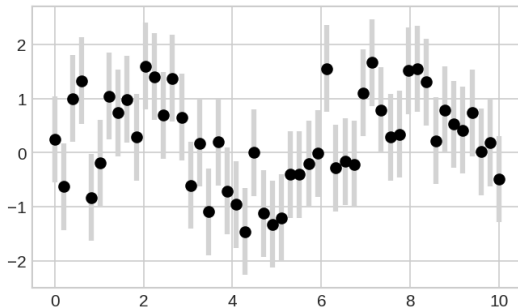
errorbar() cơ bản

```
y = np.sin(x) + dy * rng.standard_normal(50)      # du lieu "do luong" co nhieu  
ax.errorbar(x, y, yerr=dy, fmt='.k');
```



Tùy chỉnh thanh sai số

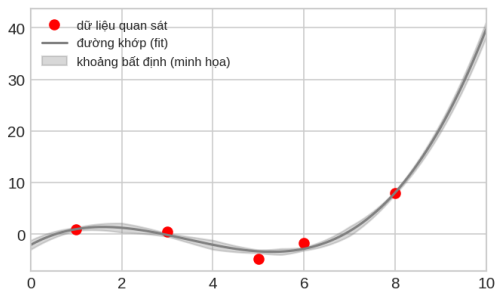
```
ax.errorbar(x, y, yerr=dy, fmt='o', color='black',  
            ecolord='lightgray', elinewidth=3, capsize=0);
```



`ecolor`: màu riêng cho thanh sai số (khác màu điểm dữ liệu) – giúp thanh sai số không "lấn át" điểm dữ liệu chính.

Dải bất định liên tục: fill_between()

```
ax.plot(xdata, ydata, 'or', label='dữ liệu quan sát')
ax.plot(xfit, yfit, '-', color='gray', label='đường khớp (fit)')
ax.fill_between(xfit, yfit - dyfit, yfit + dyfit, color='gray', alpha=0.3,
               label='khoảng bất định (minh họa)')
```



Khi mô hình dự đoán liên tục theo x (ví dụ hồi quy), `fill_between()` vẽ dải bất định (confidence band) quanh đường dự đoán – trực quan hơn nhiều so với thanh sai số rời rạc từng điểm.

Tổng kết Module 32

- `ax.errorbar(x, y, yerr=)`: thanh sai số tại từng điểm rời rạc.
- `fmt, ecolor, elinewidth, capsize`: tùy chỉnh hiển thị.
- `ax.fill_between(x, y_low, y_high)`: dải bất định liên tục – phù hợp cho đường dự đoán/hồi quy.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Dựng lưới tọa độ 2D bằng `np.meshgrid()` để tính hàm 2 biến.
- 2 Vẽ đường đồng mức bằng `ax.contour()/contourf()`.
- 3 Hiển thị mảng 2D dưới dạng ảnh bằng `ax.imshow()`.

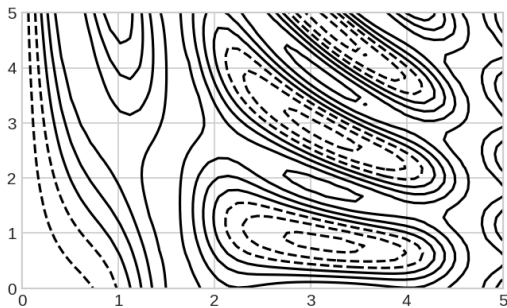
np.meshgrid() và hàm 2 biến

```
def f(x, y):  
    return np.sin(x) ** 10 + np.cos(10 + y * x) * np.cos(x)  
  
x = np.linspace(0, 5, 50); y = np.linspace(0, 5, 40)  
X, Y = np.meshgrid(x, y)      # X, Y: 2 luoi toa do 2D tuong ung tung diem  
Z = f(X, Y)                   # tinh f() dong thoi tren CA luoi (vector hoa - Buoi 2)
```

`meshgrid()` biến 2 mảng 1D (x, y) thành 2 lưới 2D để tính hàm 2 biến $f(x, y)$ trên *toàn bộ* mặt phẳng cùng lúc, không cần vòng lặp lồng nhau – vector hóa (Buổi 2) áp dụng cho bài toán 2 chiều.

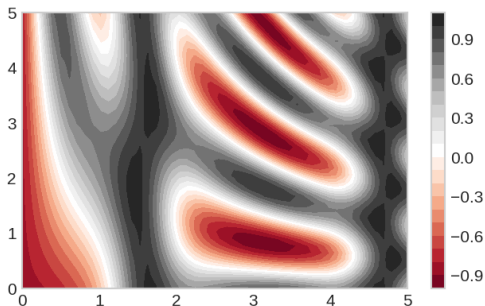
contour(): đường đồng mức

```
ax.contour(X, Y, Z, colors='black');
```



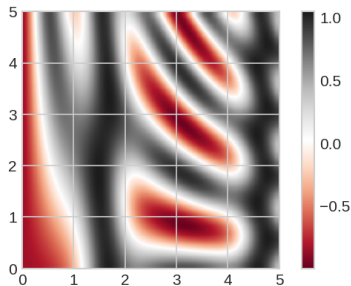
contourf(): tô màu giữa các đường đồng mức

```
cs = ax.contourf(X, Y, Z, 20, cmap='RdGy')    # 20 mức màu  
fig.colorbar(cs, ax=ax);
```



imshow(): hiển thị mảng 2D dưới dạng ảnh

```
im = ax.imshow(Z, extent=[0, 5, 0, 5], origin='lower', cmap='RdGy',  
               interpolation='gaussian', aspect='equal')  
fig.colorbar(im, ax=ax);
```



`origin='lower'`: gốc tọa độ ở góc dưới trái (mặc định ảnh có gốc ở góc trên trái);
`interpolation='gaussian'`: làm mượt giữa các ô lưới rời rạc.

Tổng kết Module 33

- `np.meshgrid()`: dựng lưới 2D để tính hàm 2 biến một cách vector hóa.
- `contour()`: chỉ vẽ đường đồng mức; `contourf()`: tô màu giữa các đường.
- `imshow()`: hiển thị mảng 2D như ảnh, cần chú ý `origin` và `extent`.
- Cả ba đều nên đi kèm `colorbar()` để đọc được thang giá trị.

Module 34: Histograms and Binnings

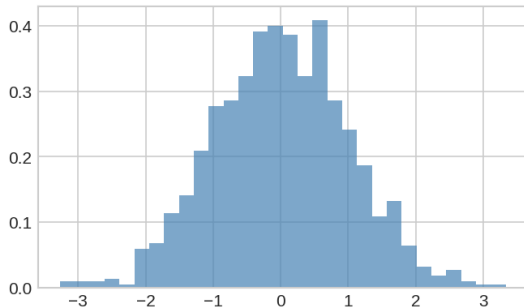
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Vẽ histogram bằng `ax.hist()` và tùy chỉnh số lượng/kiểu bin.
- 2 So sánh phân phối nhiều nhóm bằng histogram chồng, bán trong suốt.
- 3 Vẽ phân phối 2 chiều bằng `ax.hist2d()`.

hist() cơ bản

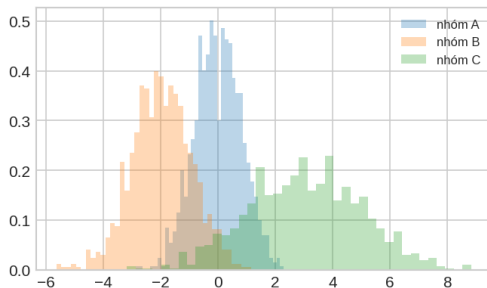
```
ax.hist(data, bins=30, density=True, alpha=0.7, histtype='stepfilled',  
        color='steelblue', edgecolor='none');
```



`density=True`: chuẩn hóa để diện tích dưới histogram bằng 1 (so sánh được với hàm mật độ xác suất), thay vì đếm số lượng thô.

So sánh nhiều phân phối

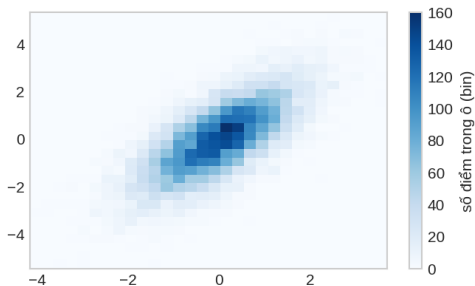
```
kwargs = dict(histtype='stepfilled', alpha=0.3, density=True, bins=40)
ax.hist(x1, **kwargs, label='nhóm A')
ax.hist(x2, **kwargs, label='nhóm B')
ax.hist(x3, **kwargs, label='nhóm C')
```



Độ trong suốt ($\alpha=0.3$) cho phép nhìn thấy các phân phối chồng lên nhau – cách trực quan hóa nhanh để so sánh 3 nhóm dữ liệu, tương tự groupby (Buổi 6) nhưng ở dạng hình ảnh.

Histogram 2 chiều: hist2d()

```
h = ax.hist2d(x, y, bins=30, cmap='Blues')  
cb = fig.colorbar(h[3], ax=ax)  
cb.set_label('số điểm trong ô (bin)')
```



Mở rộng histogram sang 2 biến đồng thời: mỗi ô lưới (bin) 2D được tô màu theo mật độ điểm rơi vào đó – hữu ích khi scatter() thông thường bị chồng lấn quá nhiều điểm để phân biệt (hexbin() là lựa chọn thay thế với ô lục giác).

- `ax.hist(data, bins=, density=)`: histogram 1 chiều cơ bản.
- `histtype='stepfilled' + alpha`: so sánh nhiều phân phối chồng lên nhau.
- `ax.hist2d()/ax.hexbin()`: histogram 2 chiều, thay thế scatter khi dữ liệu quá dày đặc.
- `np.histogram()`: chỉ tính số liệu (không vẽ) khi cần dùng số đếm cho việc khác.

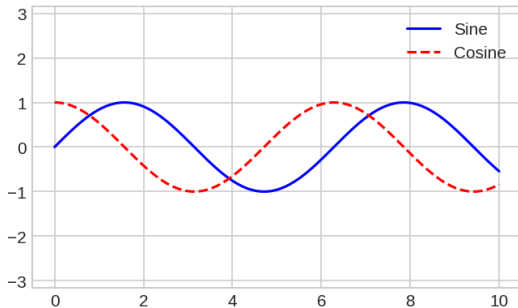
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Tạo chú giải (legend) cơ bản và tùy chỉnh vị trí, số cột.
- 2 Kiểm soát chính xác đường/nhãn nào xuất hiện trong legend.
- 3 Hiển thị 2 legend độc lập trên cùng 1 hệ trục.
- 4 Dùng legend để mã hóa *kích thước* điểm (không chỉ màu) trên dữ liệu thực tế.

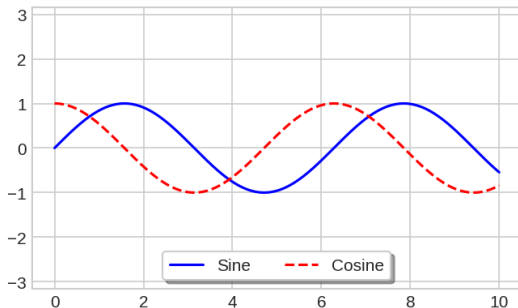
legend() cơ bản

```
ax.plot(x, np.sin(x), '-b', label='Sine')  
ax.plot(x, np.cos(x), '--r', label='Cosine')  
ax.legend();
```



Vị trí và bố cục legend

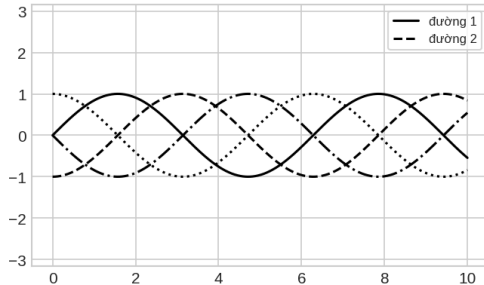
```
ax.legend(loc='lower center', ncol=2, frameon=True,  
         fancybox=True, framealpha=1, shadow=True);
```



loc: vị trí ("upper left", "lower center"...); ncol: số cột; frameon/fancybox/shadow: kiểu khung viền.

Chọn lọc đường xuất hiện trong legend

```
lines = []  
for i in range(4):  
    lines += ax.plot(x, np.sin(x - i*np.pi/2), styles[i], color='black')  
  
ax.legend(lines[:2], ['đường 1', 'đường 2'], loc='upper right'); # chỉ 2/4 đường
```

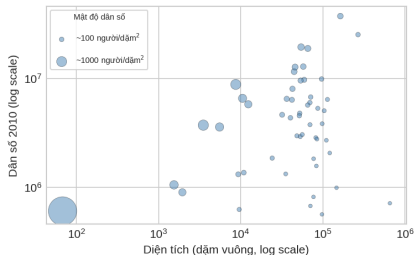


Truyền tường minh danh sách đối tượng Line2D và nhãn cho `legend()` thay vì dựa vào `label=` – kiểm soát chính xác đường nào xuất hiện, theo thứ tự nào.

Ứng dụng thật: bang Mỹ theo diện tích, dân số, mật độ

```
sizes = d2010["density"] / d2010["density"].max() * 600 + 10
ax.scatter(d2010["area (sq. mi)"], d2010["population"], s=sizes, alpha=0.5)
ax.set_xscale("log"); ax.set_yscale("log")

for area_val in [100, 1000]:      # vẽ điểm "rong" chỉ để tạo legend kích thước
    ax.scatter([], [], s=area_val/d2010["density"].max()*600+10,
                label=f"~{area_val} người/dặm$^2$")
ax.legend(title="Mật độ dân số")
```



Giải thích ví dụ: bubble chart 50 bang

Dữ liệu `state-population.csv`, `state-areas.csv`, `state-abbrevs.csv` (kho PythonDataScienceHandbook, đã dùng ở Buổi 5) – mỗi bọt biểu diễn 1 bang: trục x/y (log scale) là diện tích/dân số 2010, kích thước bọt là mật độ dân số.

Mẹo tạo legend cho kích thước

`ax.scatter([], [], s=...)` vẽ một điểm với mảng dữ liệu **rỗng** – không hiển thị trên hình nhưng vẫn tạo được một mục trong legend với kích thước mong muốn. Đây là "mẹo" tiêu chuẩn để chú thích kích thước điểm.

- `ax.legend(loc=, ncol=, frameon=...)`: vị trí và kiểu khung viền.
- Truyền (`lines`, `labels`) tường minh để kiểm soát chính xác nội dung legend.
- Có thể gọi `ax.legend()` nhiều lần với `ax.add_artist()` để giữ 2 legend độc lập.
- `ax.scatter([], [], s=...)`: mẹo tạo mục legend cho kích thước điểm.

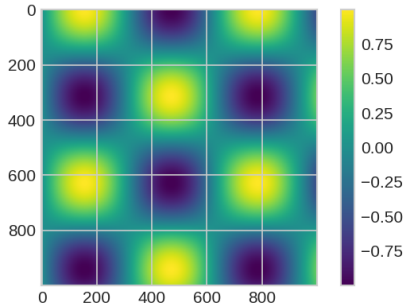
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Gắn colorbar cho hình `imshow()`/`scatter()`.
- 2 Chọn colormap phù hợp: tuần tự (sequential), phân kỳ (diverging), định tính (qualitative).
- 3 Rời rạc hóa colormap và mở rộng (extend) khi giá trị vượt ngưỡng.
- 4 Áp dụng colorbar rời rạc cho bài toán phân loại thực tế (chữ số viết tay).

colorbar() cơ bản

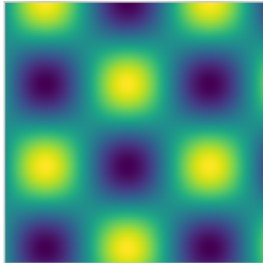
```
im = ax.imshow(I, cmap='viridis')  
fig.colorbar(im, ax=ax);
```



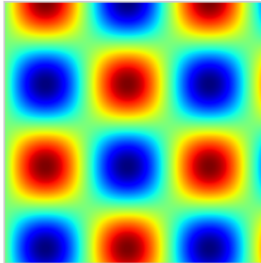
Chọn colormap phù hợp

```
for axi, cmap in zip(axes, ['viridis', 'jet', 'RdBu']):  
    axi.imshow(I, cmap=cmap)
```

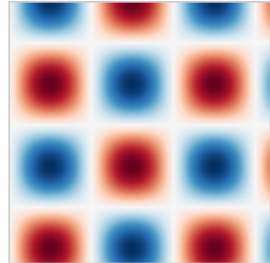
viridis



jet



RdBu

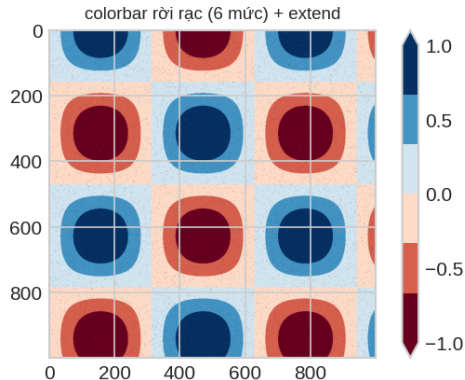
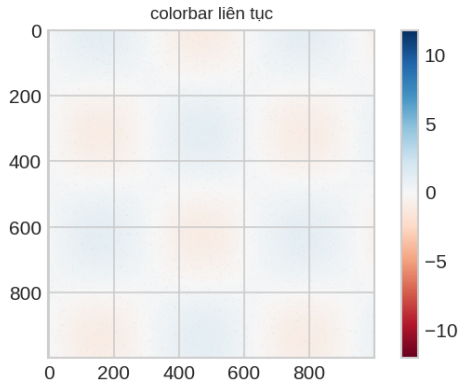


Ba họ colormap

Tuần tự (viridis): thấp–cao. **Phân kỳ** (RdBu): có tâm ý nghĩa (vd. 0). **Tránh jet**: không đơn điệu độ sáng, dễ hiểu lầm.

Colorbar rời rạc và mở rộng ngưỡng

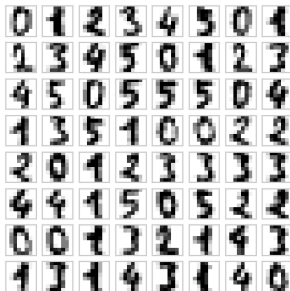
```
cmap_discrete = plt.get_cmap("RdBu", 6) # chỉ 6 mức màu rời rạc  
im = ax.imshow(I, cmap=cmap_discrete, vmin=-1, vmax=1) # mũi tên 2 đầu: giá trị vượt [-1,1]  
fig.colorbar(im, ax=ax, extend="both");
```



Ứng dụng thật: chữ số viết tay (Digits)

```
from sklearn.datasets import load_digits
digits = load_digits(n_class=6)          # ảnh 8x8 pixel, 6 chu số (0-5)

for i, axi in enumerate(ax.flat):
    axi.imshow(digits.images[i], cmap='binary')
```

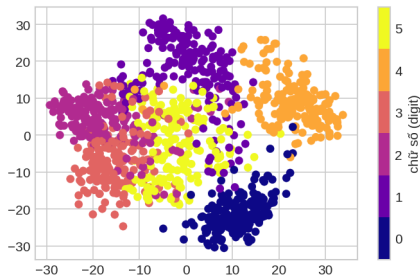


1 083 ảnh chữ số viết tay thật (8×8 pixel, thang xám), bộ dữ liệu chuẩn của Scikit-Learn – sẽ dùng lại khi học phân loại ở Buổi 11.

Colorbar rời rạc cho bài toán phân loại

```
from sklearn.decomposition import PCA
proj = PCA(n_components=2).fit_transform(digits.data)    # 64 chiều -> 2 chiều

sc = ax.scatter(proj[:, 0], proj[:, 1], c=digits.target,
                cmap=plt.get_cmap('plasma', 6))
fig.colorbar(sc, ax=ax, ticks=range(6), label='chữ số (digit)')
```



PCA (Buổi 12) nén 64 chiều điểm ảnh xuống 2 chiều để vẽ được – colorbar rời rạc 6 mức giúp phân biệt rõ 6 nhóm chữ số, dù PCA không biết trước nhãn.

- `fig.colorbar(mappable, ax=)`: gắn thang màu cho `imshow/scatter/contourf`.
- Chọn họ colormap theo bản chất dữ liệu: tuần tự, phân kỳ, định tính – tránh jet.
- `plt.get_cmap(name, N)`: rời rạc hóa colormap thành N mức; `extend`: mũi tên khi vượt ngưỡng.
- Colorbar rời rạc + `ticks=` phù hợp khi màu mã hóa *nhãn phân loại* thay vì giá trị liên tục.

Chuẩn đầu ra (Learning Outcomes)

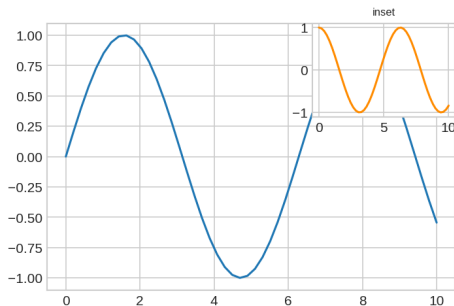
Sau module này, học viên có khả năng:

- 1 Đặt một hệ trục nhỏ lồng bên trong hệ trục lớn (`fig.add_axes`).
- 2 Tạo lưới subplots đều nhau bằng `plt.subplots()` với `sharex/sharey`.
- 3 Xây dựng bố cục bất đối xứng bằng `GridSpec`.

Axes lồng nhau (inset)

```
ax1 = fig.add_axes([0.1, 0.1, 0.8, 0.8])  
ax2 = fig.add_axes([0.62, 0.62, 0.28, 0.28])
```

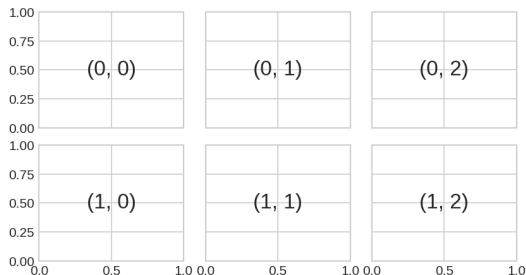
```
# [left, bottom, width, height]  
# hệ trục nhỏ, lồng bên trong
```



`add_axes([left, bottom, width, height])` nhận tọa độ *tương đối* (0 đến 1) so với Figure – cho phép đặt hệ trục ở bất kỳ vị trí/kích thước nào, kể cả lồng bên trong hệ trục khác.

Lưới subplots với sharex/sharey

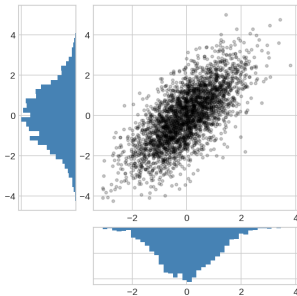
```
fig, ax = plt.subplots(2, 3, sharex='col', sharey='row')    # dùng chung trục
for i in range(2):
    for j in range(3):
        ax[i, j].text(0.5, 0.5, str((i, j)), ha='center')    # ax là mảng 2D
```



`sharex='col'`: các subplot cùng cột dùng chung trục x; `sharey='row'`: cùng hàng dùng chung trục y – tránh lặp lại nhãn trục, tiết kiệm không gian.

Bố cục bất đối xứng: GridSpec

```
grid = plt.GridSpec(4, 4, hspace=0.3, wspace=0.3)
main_ax = fig.add_subplot(grid[:-1, 1:])          # vùng chính: 3 hàng x 3 cột
x_hist = fig.add_subplot(grid[-1, 1:], sharex=main_ax) # hist dưới, dùng chung trục x
y_hist = fig.add_subplot(grid[:-1, 0], sharey=main_ax) # hist bên trái
```



GridSpec cho phép 1 subplot chiếm *nhiều ô* lưới – đây chính là bố cục "biểu đồ phân tán kèm histogram biên" (marginal histogram), rất phổ biến trong EDA (Buổi 9).

- `fig.add_axes([left, bottom, w, h])`: đặt hệ trục tự do, kẻ cả lồng nhau (inset).
- `plt.subplots(nrows, ncols, sharex=, sharey=)`: lưới subplots đều, dùng chung trục.
- `plt.GridSpec(nrows, ncols)`: bố cục bất đối xứng, 1 subplot chiếm nhiều ô lưới.
- Bố cục "scatter + histogram biên" là ứng dụng kinh điển của `GridSpec`.

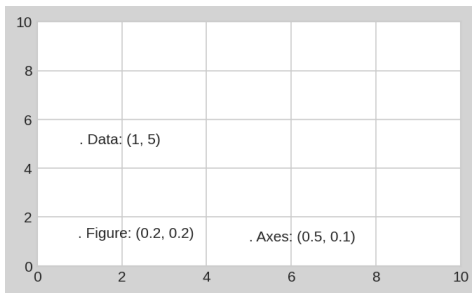
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Phân biệt 3 hệ tọa độ khi đặt văn bản: `data`, `axes`, `figure`.
- 2 Thêm chú thích văn bản đơn giản bằng `ax.text()`.
- 3 Thêm chú thích có mũi tên bằng `ax.annotate()`.
- 4 Áp dụng chú thích trên một chuỗi thời gian thực tế.

Ba hệ tọa độ khi đặt văn bản

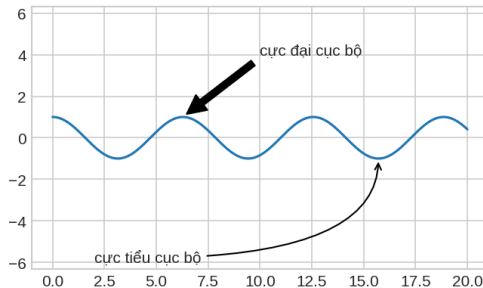
```
ax.text(1, 5, ". Data: (1, 5)", transform=ax.transData)      # theo đơn vị DỮ LIỆU  
ax.text(0.5, 0.1, ". Axes: (0.5, 0.1)", transform=ax.transAxes)  # tỉ lệ [0,1] của AXES  
ax.text(0.2, 0.2, ". Figure: (0.2, 0.2)", transform=fig.transFigure)  # tỉ lệ của FIGURE
```



`transData` (mặc định) đổi theo dữ liệu; `transAxes/transFigure` cố định theo tỉ lệ khung – hữu ích khi muốn 1 nhãn luôn ở "góc trên phải" bất kể dữ liệu thay đổi.

annotate(): chú thích có mũi tên

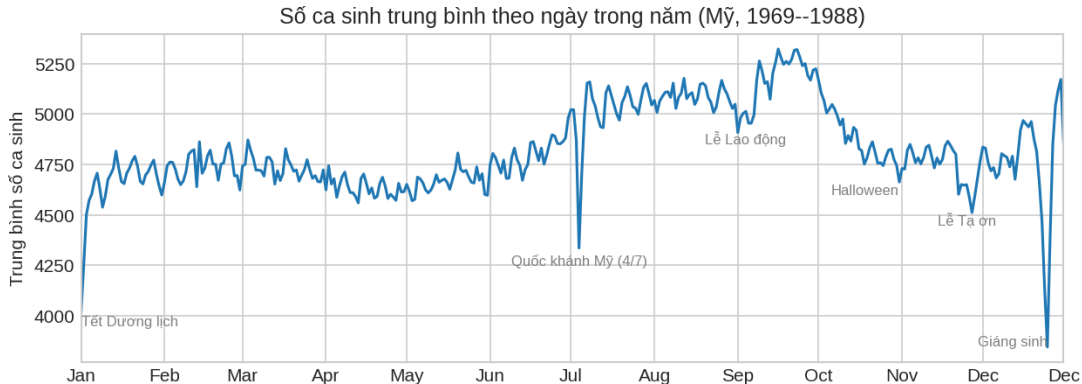
```
ax.annotate('cực đại cục bộ', xy=(6.28, 1), xytext=(10, 4),  
           arrowprops=dict(facecolor='black', shrink=0.05))  
ax.annotate('cực tiểu cục bộ', xy=(5*np.pi, -1), xytext=(2, -6),  
           arrowprops=dict(arrowstyle="->"))
```



xy: điểm được chú thích (đầu mũi tên); xytext: vị trí đặt chữ (đuôi mũi tên) – arrowprops kiểm soát kiểu mũi tên.

Ứng dụng thật: số ca sinh theo ngày trong năm

```
births_by_date.plot(ax=ax)                # trung bình số ca sinh, 366 ngày (Buoi 7)
style = dict(size=8, color='gray')
ax.text('2012-7-4', 4250, "Quốc khánh Mỹ (4/7)", ha='center', **style)
ax.text('2012-12-25', 3850, "Giáng sinh", ha='right', **style)
```



Giải thích ví dụ: births.csv có chú thích

Dữ liệu thật (CDC, births.csv, đã làm sạch ở Buổi 7: loại ngày lỗi, gộp theo tháng-ngày bất kể năm) cho thấy các **ngày lõm rõ rệt** trùng khớp với các ngày lễ lớn của Hoa Kỳ.

Vì sao? (liên hệ tư duy phản biện)

Không phải vì "ít trẻ muốn sinh vào ngày lễ" – mà vì phần lớn ca sinh mổ/khởi phát chuyển dạ theo *lịch định trước* của bệnh viện, và bệnh viện thường tránh lên lịch những thủ thuật không cấp cứu vào ngày lễ. Đây là ví dụ tốt về việc biểu đồ gợi ý *giả thuyết*, nhưng cần kiến thức lĩnh vực để *diễn giải đúng* nguyên nhân.

- 3 hệ tọa độ: `transData` (theo dữ liệu), `transAxes`, `transFigure` (theo tỉ lệ khung).
- `ax.text(x, y, "...")`: chú thích văn bản đơn giản, không có mũi tên.
- `ax.annotate("...", xy=, xytext=, arrowprops=)`: chú thích có mũi tên trỏ tới điểm cụ thể.
- Chú thích tốt biến một biểu đồ "đẹp" thành một biểu đồ *kể được câu chuyện*.

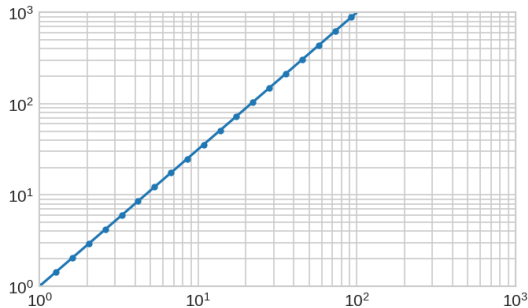
Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- ❶ Đổi thang trục sang logarit và giải thích khi nào nên dùng.
- ❷ Tùy chỉnh vị trí (Locator) và định dạng (Formatter) của tick.
- ❸ Áp dụng `plt.style.use()/plt.style.context()` để đổi giao diện tổng thể.

Thang logarit

```
ax.set_xscale('log'); ax.set_yscale('log')  
ax.set_xlim(1, 1e3); ax.set_ylim(1, 1e3)
```

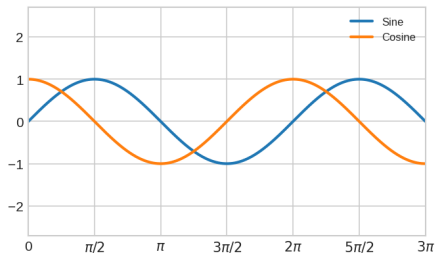


Đã dùng ngầm ở Module 35 (`legend_bubble_states.png`) – thang log phù hợp khi dữ liệu trải dài nhiều bậc độ lớn (ví dụ diện tích bang từ vài trăm đến hàng triệu dặm vuông).

Locator & Formatter tùy chỉnh

```
ax.xaxis.set_major_locator(plt.MultipleLocator(np.pi / 2))    # tick chính: bội số  $\pi/2$   
ax.xaxis.set_minor_locator(plt.MultipleLocator(np.pi / 4))    # tick phụ: bội số  $\pi/4$ 
```

```
def format_func(value, tick_number):  
    N = int(np.round(2 * value / np.pi))  
    return r"$\pi/2$" if N == 1 else (r"$\pi$" if N == 2 else f"${N}\pi/2$")  
ax.xaxis.set_major_formatter(plt.FuncFormatter(format_func))
```



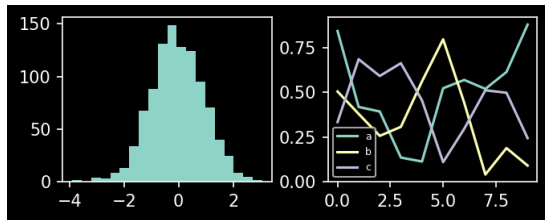
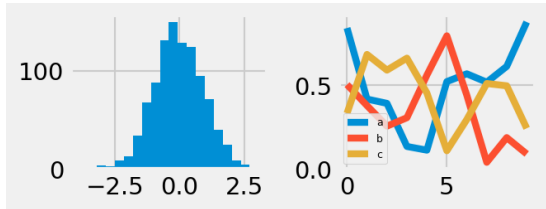
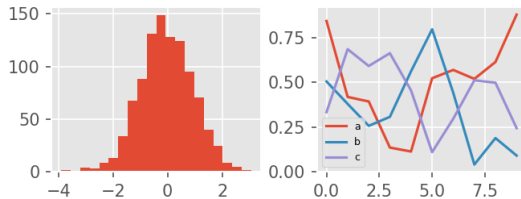
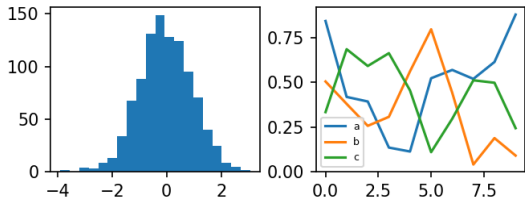
Vì sao cần tùy chỉnh tick?

Locator quyết định *vị trí* đặt tick; Formatter quyết định *chữ hiển thị* tại vị trí đó – hai khái niệm độc lập, kết hợp tự do với nhau.

- `plt.NullLocator()/NullFormatter()`: ẩn hoàn toàn tick/nhãn.
- `plt.MaxNLocator(n)`: giới hạn tối đa n tick – tránh chữ chồng chéo khi có nhiều subplot nhỏ.
- `plt.MultipleLocator(k)`: đặt tick tại bội số của k – ví dụ trang trước, bội số của $\pi/2$.
- `plt.FuncFormatter(f)`: định dạng tick bằng hàm Python tùy ý (ví dụ hiển thị π thay vì 3.14159).

Style Sheets: đổi giao diện tổng thể

```
with plt.style.context("ggplot"):  
    fig, ax = plt.subplots(1, 2)  
    ax[0].hist(data); ax[1].plot(...)
```



Giải thích: default, ggplot, fivethirtyeight, dark_background

Cùng **1 bộ dữ liệu** (đã cố định seed), chỉ đổi style – màu sắc, lưới nền, font đều thay đổi nhưng *dữ liệu và thông điệp không đổi*.

Lưu ý quan trọng

`plt.style.use(name)` đổi style cho **toàn bộ phiên làm việc** còn lại; with `plt.style.context(name)`: chỉ áp dụng **tạm thời** bên trong khối lệnh – nên dùng `context()` khi chỉ muốn 1 hình có giao diện khác biệt, tránh ảnh hưởng các hình sau đó.

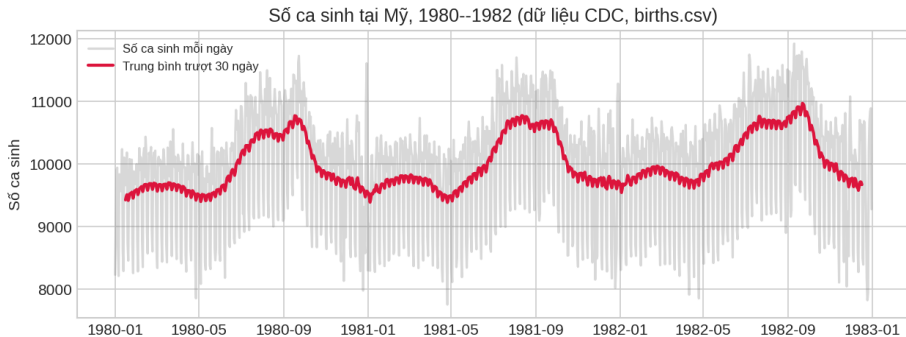
Xem toàn bộ danh sách qua `plt.style.available` (matplotlib 3.10 có hơn 25 style dựng sẵn, gồm cả các biến thể `seaborn-v0_8-*`).

Tổng kết Module 39

- `set_xscale/set_yscale('log')`: thang log, phù hợp dữ liệu trải nhiều bậc độ lớn.
- `Locator`: vị trí tick; `Formatter`: chữ hiển thị – độc lập, kết hợp tự do.
- `plt.style.use()`: đổi vĩnh viễn; `plt.style.context()`: đổi tạm thời trong khối `with`.
- Chọn style nhất quán cho toàn bộ báo cáo/đồ án giúp tăng tính chuyên nghiệp.

Ví dụ đầy đủ: xu hướng sinh 1980–1982 (kết hợp Buổi 7 + 8)

```
sub = total_full["1980":"1982"]                                # Series theo ngày (Buổi 7)
rolling = sub.rolling(30, center=True).mean()                  # trung bình trượt (Buổi 7)
ax.plot(sub.index, sub.values, alpha=0.3, color="gray", label="Số ca sinh mỗi ngày")
ax.plot(rolling.index, rolling.values, color="crimson", lw=2,
        label="Trung bình trượt 30 ngày"); ax.legend(loc="upper left")
```



Bài tập 1: Mở rộng trực quan hóa births.csv

Dữ liệu: `notebooks/data/births.csv` (đã làm sạch ở Buổi 7).

- 1 Vẽ `total.resample("YE").sum()` (tổng ca sinh theo năm, Module 30 – line plot) trên cùng 1 trục với đường xu hướng tuyến tính (dùng `np.polyfit` bậc 1) vẽ chồng lên bằng màu khác, có `legend()`.
- 2 Dùng `ax.errorbar()` (Module 32) vẽ trung bình số ca sinh theo **ngày trong tuần** kèm sai số chuẩn ($\text{std}/\sqrt{n}$) của từng nhóm.
- 3 Dùng `GridSpec` (Module 37) tạo bố cục: biểu đồ đường chính ở giữa (theo năm) + histogram phân phối số ca sinh/ngày ở bên phải.
- 4 Thử vẽ lại toàn bộ Bài tập 1 dưới `plt.style.context("fivethirtyeight")` (Module 39) – so sánh cảm nhận trực quan với style mặc định.

Bài tập 2: Trực quan hóa MovieLens & Planets

Dữ liệu: `datasets/movielens/` (Buổi 6), `seaborn.load_dataset("planets")` (Buổi 6).

- 1 Vẽ histogram (Module 34) phân phối rating trung bình theo phim (`mean_ratings`) – so sánh hình dạng phân phối giữa nam và nữ bằng 2 histogram chồng bán trong suốt.
- 2 Vẽ scatter (Module 31) `orbital_period` vs `mass` của bộ `planets`, trục log-log (Module 39), màu theo `method`, kích thước theo `distance` – dùng `cmap` định tính vì `method` là biến phân loại, không phải liên tục.
- 3 Thêm `ax.annotate()` (Module 38) chỉ ra điểm có `orbital_period` lớn nhất trên hình vừa vẽ.
- 4 *Câu hỏi thảo luận:* vì sao dùng `colormap định tính` (qualitative, ví dụ `tab10`) thay vì *tuần tự* (viridis) khi màu mã hóa `method` – điều gì sẽ gây hiểu lầm nếu dùng sai loại `colormap`?

Bảng tổng hợp các hàm Matplotlib trong Buổi 8

Nhóm	Hàm/thao tác	Mục đích
Cấu trúc	<code>plt.subplots</code> , <code>fig.add_axes</code> , <code>GridSpec</code>	Tạo Figure/Axes, bố cục nhiều hệ trục
Đường/điểm	<code>ax.plot</code> , <code>ax.scatter</code>	Line plot, scatter plot
Thống kê	<code>ax.errorbar</code> , <code>ax.fill_between</code>	Sai số rời rạc, dải bất định liên tục
2D	<code>ax.contour(f)</code> , <code>ax.imshow</code> , <code>ax.hist2d</code>	Hàm 2 biến, ảnh, histogram 2 chiều
Phân phối	<code>ax.hist</code>	Histogram 1 chiều, so sánh nhiều nhóm
Trang trí	<code>ax.legend</code> , <code>fig.colorbar</code>	Chú giải, thang màu
Chú thích	<code>ax.text</code> , <code>ax.annotate</code>	Văn bản, mũi tên chỉ dẫn
Trục & style	<code>Locator</code> , <code>Formatter</code> , <code>plt.style</code>	Tick tùy chỉnh, giao diện tổng thể

Tài liệu tham khảo I

- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed.). O'Reilly Media.
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- VanderPlas, J. (n.d.). *PythonDataScienceHandbook* [Kho mã nguồn – notebooks 04.00–04.11; dữ liệu births.csv, state-population.csv và liên quan]. GitHub.
<https://github.com/jakevdp/PythonDataScienceHandbook>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. (nguồn dữ liệu iris, digits)
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188. (bộ dữ liệu Iris)
- Centers for Disease Control and Prevention (CDC). *U.S. births by date* [Bộ dữ liệu] (dẫn qua notebook 03.09/03.11/04.09 của PythonDataScienceHandbook).
- The Matplotlib development team. *Matplotlib documentation*. <https://matplotlib.org>

Chuẩn bị cho buổi tiếp theo: Trực Quan Hóa Dữ Liệu II & EDA Thực Hành

Thông điệp

Matplotlib là nền tảng – Buổi 9 xây dựng tiếp bằng Seaborn (giao diện cao cấp, thống kê tích hợp sẵn), trực quan hóa 3D, dữ liệu địa lý, và một quy trình EDA *end-to-end* hoàn chỉnh trên case study thực tế (US Baby Names, MovieLens). Đây cũng là bước chuẩn bị trực tiếp cho Đồ án giữa kỳ (Buổi 9).